Sistema digital de controle de estoque

Projeto individual — IFAL Arapiraca

A necessidade: organização e controle

Organizar materiais e a estrutura interna do estabelecimento.

Saber onde cada item está guardado, sem endereços duplicados.

Controlar entradas, saídas e itens emprestados para eventos.

Objetivo do projeto

Tornar eficiente e 100% digital
toda a organização interna
do estoque da empresa.

O sistema reúne localização,
movimentações e controle de consumíveis.

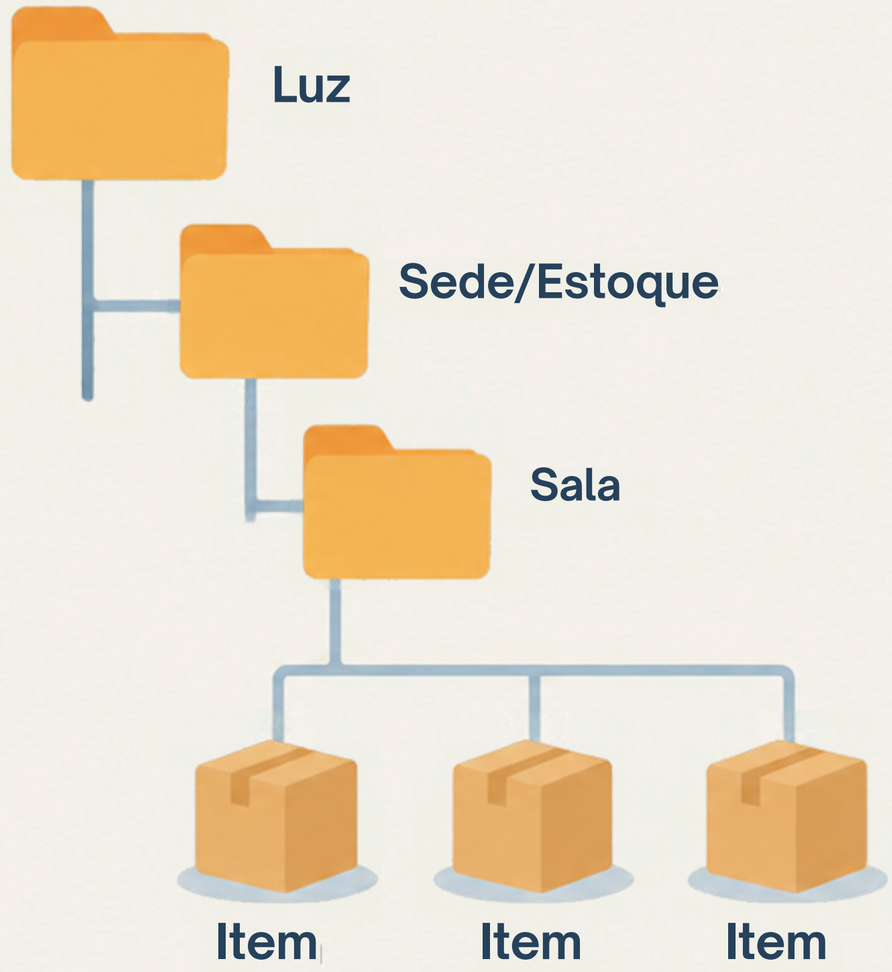
Organização dos itens

1. Organizar em Luz > Sede/Estoque > Sala > Item.

2. Cadastrar cada item com nome, local e sala.

3. Mover itens para outra sala ou local quando permitido.

4. Manter um único endereço para cada item.



Itens em eventos

Marcar os itens e informar o evento ao retirá-los.

Devolver os itens ao final do evento.

Consultar o que ainda está fora e em qual evento.

Controle de consumíveis

Cadastrar consumíveis da sede e do estoque.

Registrar entradas e saídas sem permitir saldo negativo.

Exibir aviso quando um consumível atingir o mínimo.

Registro e segurança

- Registrar data, responsável e ação em cada movimentação.

- Proteger a consulta do registro com PIN e até três tentativas.

- O PIN é único e fica no código; a versão final teria login por pessoa.

Requisitos não funcionais

Usabilidade:

uma tela com botões e pastas, ainda sem validação com a equipe.

Portabilidade:

Python com tkinter e json.

Confiabilidade:

os dados são salvos a cada ação.

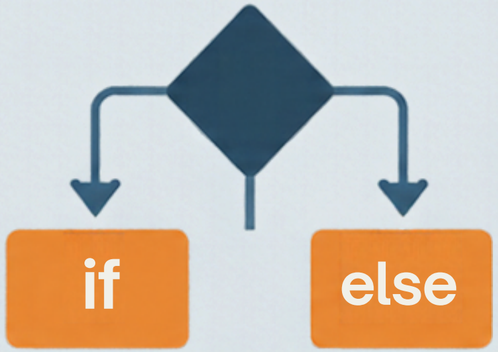
Desempenho:

meta de resposta em até um segundo com mil itens; ainda não medida.

Estruturas utilizadas no código

Decisão:

if/else verifica estado
do item, saldo de consumível
e PIN.



Repetição:

while limita tentativas
do PIN; for percorre itens,
salas e consumíveis.

Organização:

funções separam tarefas;
listas guardam coleções;
dicionários nomeiam campos.

Dados:

tuplas agrupam informações;
strings formam mensagens;
arquivos JSON salvam e
carregam os dados.

Quais e onde foram utilizadas as estruturas no código? (2,5 pontos)

Estrutura	Onde (linhas)	Para que serve no sistema
if/else	retirar_itens (54 a 56). Também 93 e 95, 105 e 108, 302 e 324	Decide se o item já está em evento, se o consumível está acabando e se o PIN está certo.
while	botao_registro (320)	Dá até 3 tentativas para digitar o PIN.
for	30 (carregar), 52 (retirar_itens), 64, 75, 87, 89 e 91 (desenhar_itens), 102, 120	Percorre itens, salas e consumíveis. Os três for encaixados formam as pastas.
Subprogramas	26 funções: 23 def normais e 3 dentro de outras (confirmar nas linhas 143 e 189 e criar_sala, na 180). Ex.: retirar_itens (49 a 60) e pedir_local (215)	Cada tarefa é uma função, e os botões só chamam as funções.
Listas	15 (dados), 11 a 14 (LOCAIS), 51 (ja_fora), 74 (linhas), 119 (marcados), 325 e 326 ([-20:], .reverse()), 187 (LOCAIS[local].append, sala nova)	Guardam vários itens, consumíveis, salas e registros.
Strings	44 (junta com +), 59 e 70 (f-strings), 183 (.strip()), 184 (.casefold()), 80, 261 e 327 (.join), 121 (.startswith)	Montam o registro, limpam o que a pessoa digita e formatam mensagens.
Dicionários	11 a 14 (LOCAIS), 15 (dados), 16 (usuario), 31 (ficha do item), 133 e 169 (resultado)	Cada item é um registro com campos nomeados: nome, local, sala, status e evento.
Tuplas	21 e 22 (open e json.load), 24 (lê as salas salvas), 38 e 39 (open e json.dump)	Dados de exemplo e função que devolve duas informações juntas.
Arquivos	21 e 22 (open e json.load), 35 e 36 (open e json.dump)	Lê o arquivo ao abrir e grava a cada ação.

Resultados e limites

Atendido:

localização dos itens, saídas para eventos, devoluções e consulta do que está fora.

Atendido:

controle de consumíveis e prevenção de movimentações duplicadas.

Parcial:

registro protegido por PIN; a solução final precisaria de login individual.

Pendente:

validar a facilidade de uso com a equipe e permitir vários usuários simultâneos.

Conclusão

O sistema organiza materiais, registra movimentações e torna o controle do estoque mais claro.

Próximos passos:
validar o uso com a equipe e evoluir o acesso por usuário.